```
What is your app name? · osfy-tauri
What should the window title be? · Sample Tauri Application
Where are your web assets (HTML/CSS/JS) located, relative to the "<current dir>/src-tauri/tauri.conf.json" file that will be created? · ../frontend
What is the url of your dev server? · ../frontend
What is your frontend dev command? ·
What is your frontend build command? ·
```

Figure 1: Queries asked during initialisation

Developing a Tauri app

Tauri supports almost every web frontend framework. If you are already proficient in any of the web frontend frameworks, you can easily create a GUI application with minimal effort. Let's see how you can transform your simple web development skills into software development skills.

As I mentioned before, Tauri consists of two parts:

- A Rust layer, which takes care of OS-level functionality (let's call it the backend)
- A frontend built using web technologies like HTML/CSS/JS

You can set up your Tauri project easily using Tauri CLI. To install Tauri CLI you can use Rust's package manager, Cargo or Node.js' package manager, npm.

To use Cargo, type:

```
cargo install tauri-cli
cargo create-tauri-app
```

To use npm, give the following command:

```
npm install --save-dev @tauri-apps/cli
```

After installing Tauri CLI, you can create a directory/folder in which you develop your Tauri application. Let's call it 'osfy-tauri', for example. Inside the folder, you can create another folder called 'frontend' to keep your UI related files.

Create a simple *html* file inside the 'frontend' folder; let's name it *index.html*.

The content of *index.html* could be something like this:

```
<html>
<head>
    <title>My First Tauri Application</
title>
</head>
<body>
    <h1>Hello OSFY readers!</h1>
</body>
</html>
```

Now go back to your parent folder, and in the terminal, write:

```
cargo tauri init
```

This command will guide you through setting up your directory as a Tauri project.

The third and fourth questions in Figure 1 are related to where the frontend assets of your application will be located. Since we have created a simple *html* file, we can use its directory location, which is '../frontend'. You can also leave the frontend *dev* and *build* command as empty, since we are not using any node project or framework but a simple HTML file for the frontend. If you are using a frontend framework, you can simply use the *build* and *dev* command in the *package.json* file.

After the command is executed, your directory structure will look like what's shown in Figure 2.

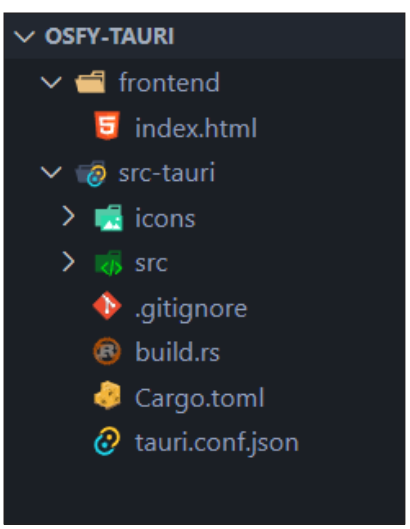

Figure 2: Tauri project directory structure

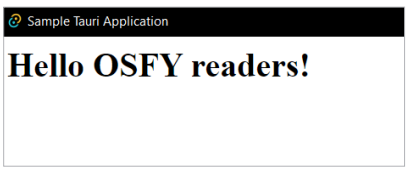

Figure 3: Our first Tauri application screen

The 'src-tauri' generated by Tauri CLI consists of your backend related source code, which is a Rust program. Now let's try running the application using the command:

```
cargo tauri dev
```

You will be greeted with an application screen, as shown in Figure 3.

Calling Rust functions from frontend

Tauri provides a straightforward way to call Rust functions from the frontend. These are called *commands*. Tauri commands enable the frontend to use the OS for any resource-heavy task instead of relying on a browser, as done in Electron.

Let's see how commands work. Make changes to *index.html* so that you use a button to call a function that invokes Rust functions, using the 'invoke' function provided by Tauri to the JS front.

Now, go to the *src-tauri/src/main.rs* file and write a corresponding Rust function that you would like to call. This function will return a string from Rust to JS. Additionally, you need to register this function by adding it to your Tauri app using the *invoke_handler* function.

Next, go to *src-tauri\tauri.conf.json*, Tauri's configuration file. Add what is given in Figure 6 under the *build* section.

When '*withGlobalTauri*' is set to 'true', Tauri's API methods are available globally in the application.